

Funções de strings

Getch()

- O `getch()` é uma função que lê o input do utilizador quando pressionado uma tecla. Não é necessário um especificador, apenas atribui-se a função ao valor variável em questão.

```
char letra;  
printf("Pressione uma Tecla:\n");  
letra = getch();
```

Exercício - getch()

- Crie um menu que caso o utilizador pressione a tecla 'Esc' saia do menu, crie também dois cases. O menu deve usar a função **getch()** e ser limpo e organizado.

Gets()

- O `gets()` é uma função que lê o input do utilizador, tratando-a como uma string. Utiliza-se “%s”. Com isso é possível manipular a variável para criar programas.

```
char name[50];  
printf("Type your name: ");  
gets(name);
```

Exercício - gets()

- Crie um programa que peça ao utilizador para inserir uma frase. Leia o input do utilizador e use a função **gets()**. Utilize obrigatoriamente um ciclo “for”, inverta a frase e imprima novamente.

Scanf()

- Em C, `scanf()` é uma função da biblioteca `stdio.h` que é utilizada para ler dados de input a partir do teclado. A função é projetada para analisar e armazenar os valores inseridos pelo utilizador.

```
int years;  
printf("How old are you: ");  
scanf("%d", &years);
```

Scanf()

- Também se pode utilizar o `scanf()` para variáveis do tipo string, mas quando esse é o caso, não se deve utilizar o endereço de memória.

```
char name[50];  
printf("type your name: ");  
scanf("%s", name);
```

Exercício - scanf()

- Faça um código que peça ao utilizador para inserir a sua idade e o seu nome. No final, o programa deve mostrar o ano que o utilizador nasceu e o seu nome.

Getche()

- O **getche()** é uma função de input imediata, onde é pedido ao utilizador um ou mais caracteres, onde não é preciso clicar enter, que é o que diferencia esta do **getchar()**. O **getche()** faz echo à tecla seleccionada pelo utilizador.

```
printf("Escreva um caracter: ");  
char che = getche();
```

Exercício - `getche()`

- Crie um programa que pede ao utilizador para inserir uma palavra passe usando o **`getche()`**, exibindo no final a password.

Getchar()

- O `getchar()` é uma função fundamental em C que faz parte da biblioteca `<stdio.h>` que lê um único carácter a partir da entrada padrão, normalmente o teclado. Devolve um número inteiro que representa o valor do carácter lido.

```
char ch;  
printf("Enter a character: ");  
ch = getchar();
```

Exercício - getchar()

- Escreva um programa que peça ao utilizador para introduzir uma frase e depois conte o número de vogais (a, e, i, o, u) nessa frase usando o `getchar()`. Imprima a contagem total de vogais.

Strdup()

- A função `strdup()` faz parte da biblioteca `<string.h>`, é usada para duplicar (copiar) uma string. Ela aloca dinamicamente memória suficiente para armazenar uma cópia da string fornecida e, em seguida, copia a string original para a nova área de memória.

```
char original[] = "Hello word!";  
char *duplicate = strdup(original);
```

Strdup() e strcpy()

- Enquanto a função **strcpy()** requer que a string de destino tenha espaço suficiente para acomodar a string original. Pode levar a problemas se o espaço não for suficiente, pois não há verificação automática, já a função **strdup()** aloca dinamicamente memória suficiente para armazenar uma cópia da string fornecida.

Exercício - strdup()

- Desenvolva um código em C que solicite ao utilizador um nome e depois, usando a função **strdup()** copie a mensagem para outra variável, mostrando a mensagem original e a copiada.

Desafio

- Escreva um programa em C onde o utilizador tem de criar uma conta, com nome e palavra pass, depois tem de selecionar o tipo de plano que quer. Por fim deve ser duplicadas as credenciais do utilizador para que possam ser guardadas nas variáveis **dupName** e **dupPass**.

Strtok()

- A função `strtok()` em linguagem C que é utilizada para dividir uma string em tokens(substrings).
- Para declarmos a função usamos:

```
char *strtok(char *str, const char *delim);
```

Strtok() - Explicação

- **Str:** É um ponteiro para a string que será dividida. Na primeira chamada, esta string deve ser especificada.
- **Delim:** Uma string contendo caracteres que servirão como delimitadores. A strtok() usará esses caracteres para identificar onde a string deve ser dividida em tokens.

Strtok() - Exemplo

```
char frase[] = "Ola, mundo! Bem-vindo ao strtok";  
char delimitadores[] = " , !.";
```

```
char *token = strtok(frase, delimitadores);
```

```
While(token != NULL)  
{  
    printf("Token: %s\n", token);  
    token = strtok(NULL, delimitadores);  
}
```

Strtok() - Desafio

- Crie um programa em C que solicita ao utilizador para inserir uma frase. Utilize a função **strtok()** para analisar a frase e exibir cada palavra em uma nova linha.

Snprintf()

- A função **snprintf()** é utilizada para formatar uma sequência de caracteres e armazená-la em um buffer. Ela é uma versão segura da função **printf()**, pois aceita um argumento adicional para especificar o tamanho máximo do buffer, evitando assim possíveis estouros de buffer.

```
int snprintf(char str, int size, const char format, ...);
```

Snprintf() - Explicação

- **Str:** Um ponteiro para o buffer onde a string formatada será armazenada.
- **Size:** O tamanho máximo do buffer, incluindo o caractere nulo do terminal ('\0').
- **Format:** A string de formato, semelhante à utilizada em printf(), que especifica como os argumentos adicionais devem ser formatados.
- **...** : Argumentos adicionais a serem formatados de acordo com a string de formato.

Snprintf() - Exemplo

```
char buffer[50];  
int numero = 42;  
  
snprintf(buffer, sizeof(buffer), "O número é: %d", numero );
```

Snprintf() - Desafio

- Crie um programa em C que solicita ao utilizador inserir seu nome e idade. Utilize a função **snprintf()** para formatar esses dados em uma frase e exibi-la de forma segura, evitando possíveis estouros de buffer.

Fflush()

- **Fflush(stdin)** é uma prática frequentemente utilizada por programadores em C na tentativa de limpar o buffer de entrada associado ao fluxo padrão de entrada (stdin). A função **fflush** é originalmente projetada para ser usada com fluxos de saída, não com fluxos de entrada.

Fflush() - Exemplo

```
int numero;  
  
printf("Digite um número inteiro: ");  
Scanf("%d", &numero);  
  
fflush(stdin);  
  
printf("Número lido: %d\n", numero);
```

Fflush() - Desafio

- Crie um programa em C que solicita ao utilizador inserir uma palavra e, em seguida, um número inteiro. Utilize **fflush(stdin)** entre as duas leituras para garantir que o buffer de entrada seja limpo corretamente.

Strncpy()

- A função **strncpy()** é uma abreviação de string (conjunto de caracteres) que copia até n caracteres de uma string.
- **Strncpy** é uma função da biblioteca string.h

```
//Escreve - se sempre primeiro o destino depois a origem:  
strncpy(target, source, 5);
```

Strncpy() - Desafio

- Peça ao utilizador uma string que obrigatoriamente tem de ter no mínimo 10 caracteres e no output final irá mostrar apenas os 5 primeiros caracteres da string.

Strspn()

- A função **strspn()** em C recebe duas strings como argumentos. A função vê quais letras da primeira string são iguais a da segunda string e termina quando encontra uma letra que não seja igual.

```
char str1[] = "banana";  
char str2[] = "bananas";  
  
int palavra = strspn(str1, str2);
```

Strspn() - Desafio

- Escreva um programa em C que compare duas strings e encontre a posição da primeira letra diferente entre elas. Utilize a função **strspn()** para realizar essa comparação.

Strcmpi()

- A função **strcmpi()** não diferencia as letras maiúsculas das letras minúsculas e a função **strcmp()** consegue diferencia-las. Se as strings forem iguais, a função retorna 0. Se a primeira string for **menor** que a segunda, ela retorna um valor negativo. Se a primeira string for **maior** que a segunda, retorna um valor positivo.

```
int result = strcmpi(str1, str2);
```


Strcmpi() - Desafio

- Cria um programa em C que peça ao utilizador para escrever duas strings, e que no final diga por ordem alfabética qual string vem primeiro.

Strrchr()

- A função `strrchr()` é uma função em C que encontra a última ocorrência de um caracter na frase e mostra a frase a partir deste caracter. Se o caracter não estiver na string o resultado é NULL.

```
Char *resultado = strrchr(str, ch);
```

Strrchr() - Desafio

- Faça um programa em C, utilizando a função **strrchr()**, que peça uma frase ao utilizador e de seguida peça um caracter e que veja se o caracter inserido está na frase.

Strstr()

- A função **strstr()** em C que encontra substrings, ou seja uma string dentro de outra, sendo que a maior string seja apresentada primeiro, pois é lá que vai estar a nossa substring. Se a string não estiver na string maior o resultado é NULL.

```
Char *resultado = strstr(str1, str2);
```

Strstr() - Desafio

- Faça um programa em C, utilizando a função **strstr()**, que peça uma frase ao utilizador e que peça uma palavra e que veja se a palavra inserida está na frase.

Strcmpi()

- O **strcmpi()** é como o **strcmp()**, mas o **strcmpi()** não é a sensível a letras minúsculas ou maiúsculas, pois o **strcmpi()** põe os seus argumento em minúsculas. O **strcmpi()** dá: -1 se a primeira string formenor que a segunda, 0 de zero diferenças e +1 se a primeira string for maior que a segunda.

```
int result = strcmpi(str1, str2);
```

Strcmpi() - Desafio

- Faça um código que peça ao utilizador para definir uma senha e depois confirma a senha, se as senhas forem iguais printar: “Senha correta. Acesso permitido.” caso contrário printar: “Senha incorreta. Acesso negado.”

Sprintf()

- A função **sprintf()** em C é usada para formatar e armazenar uma sequência de caracteres no buffer. Isso significa que podemos criar sequências de caracteres formatados sem as imprimir diretamente no ecrã.

```
sprintf(buffer, "O valor de x é: %d", x);
```


Sprintf() - Desafio

- Escreva um programa em C que solicite ao utilizador que insira seu nome, idade e cidade. Em seguida, o programa deve usar a função **sprintf()** para formatar essas informações em uma única string e imprimi-la na tela.

Strchr()

- A função **strchr()** é uma função da linguagem de programação C que é utilizada para encontrar a primeira ocorrência de um carácter específico numa sequência de caracteres.

```
char *posicao = strchr(str, 'o');
```

Strchr() - Desafio

- Escreva um programa em C que peça ao utilizador que digite uma palavra e um caractere. O programa deve então verificar se o caractere digitado pelo utilizador está presente na palavra e informar a posição do caractere, se encontrado, ou exibir uma mensagem indicando que o caractere não está presente na palavra.

Strncat()

- A função **strncat()** em C é utilizada para concatenar (juntar) duas strings. O nome "strncat" vem de "string concatenate". Essa função recebe dois argumentos: a string de destino, que é onde a concatenação será realizada, e a string de origem, que será adicionada à string de destino.

```
// Concatena no máximo 5 caracteres de origem  
strncat(destino, origem, 5);
```

Strncat() - Desafio

- Considere a seguinte situação: estás a desenvolver um programa que pede ao utilizador o seu nome e apelido separadamente e, depois, cria e mostra um nome completo concatenando apenas os primeiros três caracteres do apelido no final do nome.

IsAlpha

- A função **isAlpha()** verifica se um caractere é uma letra do alfabeto (A a Z ou a a z) e retorna verdadeiro se for, senão, retorna falso.

```
#include <ctype.h>

char caractere = 'A';

if (isalpha(caractere)) {
    printf("%c é uma letra do alfabeto.\n", caractere);
} else {
    printf("%c não é uma letra do alfabeto.\n", caractere);
}
```